A8 Incremental processing

Auto Loader, change data feed and processing only new waves.

A8-L1 Process only the waves you have not processed, correctly

Same filters on both sides, an explicit anti join, a first-run branch, and an idempotent load behind it.

```python
def unprocessed_waves(spark, bronze, silver, tracker_id) -> list[str]:
    source = spark.table(bronze).filter(F.col("tracker_id") == tracker_id).select("wave_id").distinct()
    done = spark.table(silver).filter(F.col("tracker_id") == tracker_id).select("wave_id").distinct()
    return sorted(r.wave_id for r in source.join(done, "wave_id", "left_anti").collect())
```

Rule for incremental wave detection	Filter bronze and silver by the same keys, such as tracker_id, then left-anti-join.
Join type for 'in A but not in B'	left_anti.
First run, silver missing	spark.catalog.tableExists(silver) is False: every bronze wave is new.
What does wave-ID detection miss?	Redeliveries of waves silver already holds. Use ingestion timestamps, change data feed or Auto Loader.

A8-L2 Auto Loader for file ingestion into bronze

Auto Loader replaces a fragile processed-files list with a checkpoint that commits together with the Delta write.

```python
(
    spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("cloudFiles.schemaLocation", schema_path)
    .option("header", "true")
    .load(landing_path)
    .select("*", F.col("_metadata.file_path").alias("source_file"))
    .writeStream
    .option("checkpointLocation", checkpoint_path)
    .trigger(availableNow=True)
    .toTable("research_prod.research_bronze.nat_raw")
)
```

What does Auto Loader's checkpoint store?	Discovered files (in RocksDB) and stream progress, so each file is processed exactly once across restarts.
Default schema evolution mode without a supplied schema	addNewColumns: stop with UnknownFieldException, record the new schema, restart.
Incremental at batch cost	.trigger(availableNow=True) on a scheduled job.
Checkpoint rule	One per stream, in durable storage such as a UC volume. Never shared, never deleted casually.

A8-L3 Change data feed, so downstream layers read only what changed

Read after the last version, drop pre-images, keep the newest change per key, and recompute only what the changes touch.

```python
ALTER TABLE research_prod.research_silver.nat_respondents
```

```
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

```
changes = (
    spark.read.option("readChangeFeed", "true")
    .option("startingVersion", last_version + 1)
    .table("research_prod.research_silver.nat_respondents")
)
```

Turn on change data feed	ALTER TABLE t SET TBLPROPERTIES (delta.enableChangeDataFeed = true)
The four _change_type values	insert, update_preimage (old), update_postimage (new), delete.
Two rules for consuming CDF	Drop update_preimage. Keep the newest change per key by _commit_version.
Where does the next run start?	startingVersion = last processed version + 1 (it is inclusive), or a streaming checkpoint.